



HAUTE ÉCOLE
D'INGÉNIERIE ET DE GESTION
DU CANTON DE VAUD
www.heig-vd.ch



Laboratoire 3 : Linux et Xenomai EVL

Département : Technologies de l'Information et de la Communication (TIC)
Unité d'enseignement : Programmation Temps Réel (PTR)

Auteurs : Fabien Leger & Arnaut Leyre
Professeur : Brunet Yorick
Assistant : Miceli Jean-Pierre
Classe : PTR-A
Date : 30 avril 2026

1 Introduction

Ce laboratoire a pour objectif d'analyser le comportement de tâches périodiques sous Linux dans un contexte temps réel. Nous avons mesuré la régularité d'une tâche de 10 ms à l'aide d'un GPIO et d'un oscilloscope.

Plusieurs approches ont été testées : timer avec signaux, thread POSIX temps réel et thread EVL, afin de comparer leurs performances.

2 Exercice 1 : signal_timer2.c Mesures de performance

Expliquez comment le programme gpio_toggle.c fonctionne ?

Le programme `gpio_toggle.c` commence par initialiser l'accès aux registres matériels de la carte DE1-SoC grâce à la fonction `init_de1soc_io()`.

Il lit ensuite l'état des interrupteurs (switches) via `read_switch()` et reporte cette valeur sur les LEDs à l'aide de `write_led(switch_val)`. Cette étape constitue une vérification simple permettant de s'assurer que l'accès aux périphériques mémoire-mappés fonctionne correctement.

Dans un second temps, le programme configure une broche GPIO en sortie, puis fait osciller son état périodiquement toutes les 10 ms à l'aide d'un thread POSIX. On peut brancher l'oscilloscope pour analyser le signal de sortie.

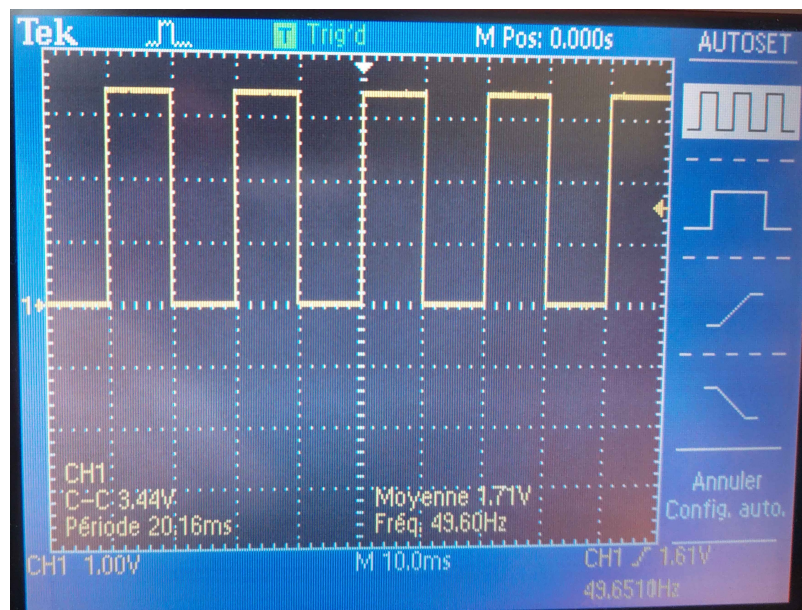


FIGURE 1 – Vue oscilloscope

Le signal obtenu est un signal carré, observable à l'oscilloscope, avec une période d'environ 20 ms (soit une fréquence de 50 Hz).

Enfin, de légères variations (jitter) peuvent apparaître. Elles sont dues à l'utilisation de la fonction `nanosleep()` ainsi qu'à l'ordonnancement du système Linux, qui ne garantit pas un comportement strictement temps réel.

Modifiez `signal_timer.c` pour contrôler le port GPIO

Fait dans le fichier `signal_timer.c`.

Expliquez les résultats obtenus avec l'oscilloscope et le nouveau programme `signal_timer.c`

On obtient les résultats suivants.

Oscilloscope

— Période : 20.10ms

— Fréquence : 49.6Hz

`signal_timer2.c` (version avec GPIO)

```
evl-de1 ~ # ./signal_timer2 100
9990250
9997830
9999060
9999380
9998740
10000070
9999900
9999470
9999750
10001240
# ...
```

La tâche périodique est activée toutes les 10 ms, ce qui correspond aux mesures obtenues avec `clock_gettime()`. Cependant, le GPIO est inversé à chaque activation. Un cycle complet du signal sur l'oscilloscope nécessite donc deux activations successives de la tâche. La période observée à l'oscilloscope est donc d'environ 20 ms, soit une fréquence d'environ 50 Hz. Cela est cohérent avec une tâche exécutée toutes les 10 ms.

3 Exercice 2 : Environnement chargé

Comparez vos résultats avec les résultats précédent.

```
taskset -pc 0 $$  
stress --cpu 8 --io 4 --vm 1 --timeout 10s
```

- --cpu 8 → 8 threads qui font des calculs intensifs
- --io 4 → 4 threads qui font des opérations I/O
- --vm 1 → 1 thread qui fait des allocations mémoire
- --timeout 10s → durée = 10 secondes

```
evl-de1 ~ # ./signal_timer2 100  
9692760  
10834740  
9160910  
10034580  
9961210  
16567570  
3435980  
16560340  
3438960  
10038480
```

On observe qu'en soumettant les processeurs à une charge intensive, le programme `signal_timer2.c` présente des variations de latence lors des mesures. Par exemple, une période peut atteindre 16.5 ms, suivie d'une période plus courte d'environ 3.5 ms.

Ce comportement indique que la mesure intermédiaire a été retardée d'environ 6.5 ms. Ce retard s'explique par le fait que le processeur était occupé par d'autres tâches, retardant ainsi l'exécution de notre programme chargé de gérer le signal.

Cela permet de se rendre compte de la différence d'un programme seul versus un programme entouré d'un ensemble d'autres programmes ce qui rend le tout beaucoup plus complexe. L'importance d'un ordonnancement bien choisi est souligné par cette expérience.

On peut également regarder les effets sur l'oscilloscope. À causes des différences entre l'envoi des données sur le port GPIO, la fréquence affichée change constamment et il n'y a pas de signal carré clair comme à l'exercice 1 2.

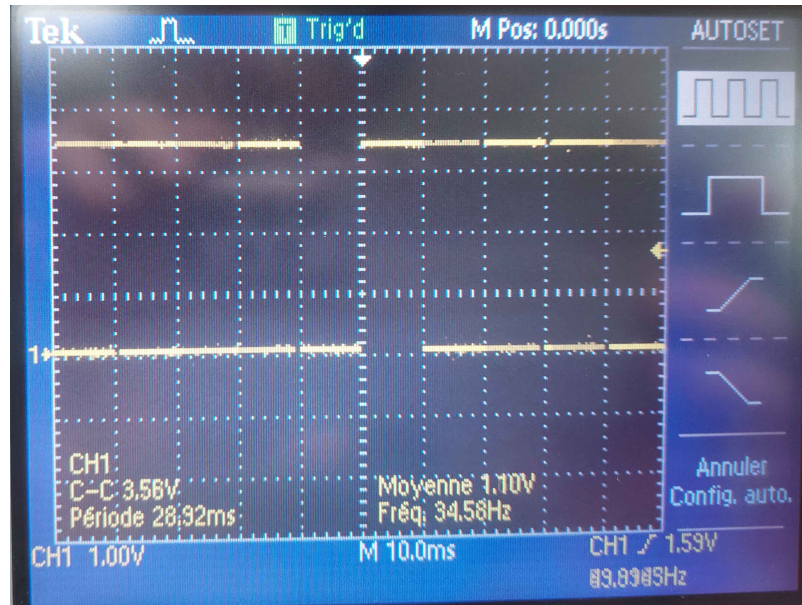


FIGURE 2 – Vue oscilloscope low

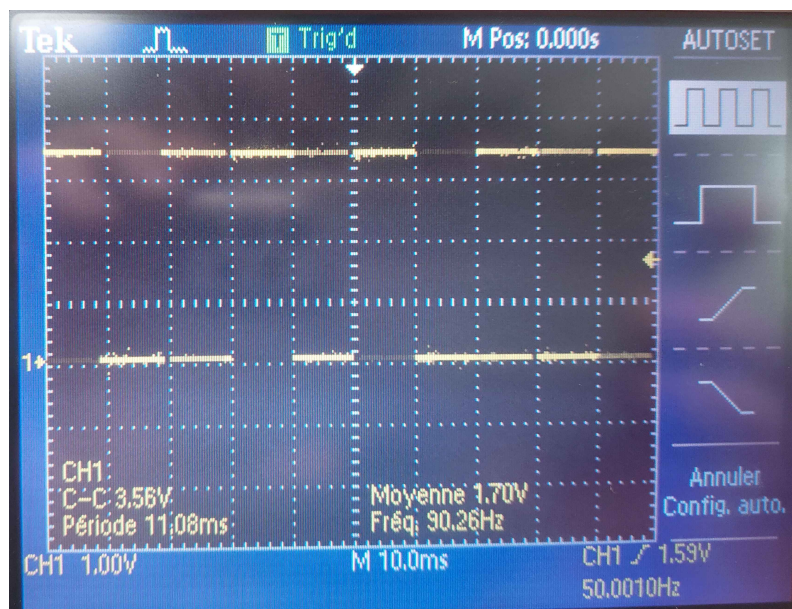


FIGURE 3 – Vue oscilloscope high

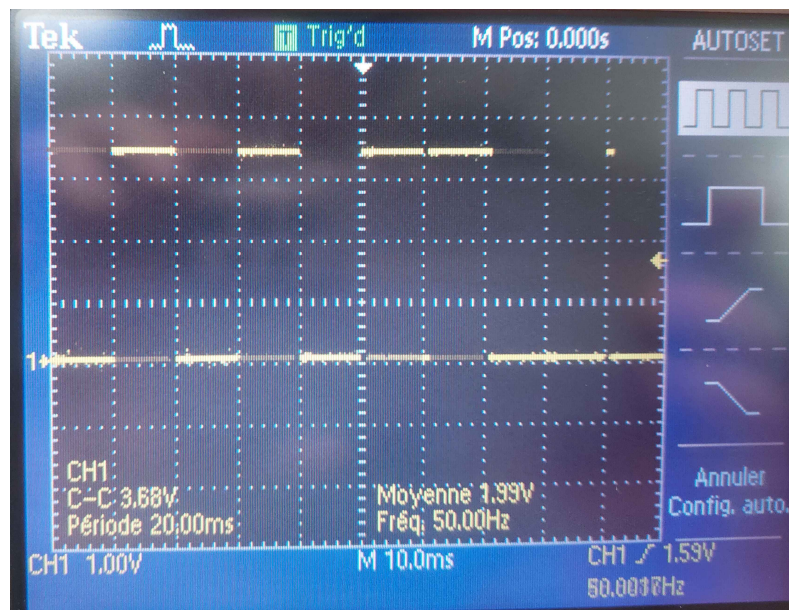


FIGURE 4 – Vue oscilloscope median

En regardant les captures d'écran de l'oscilloscope comme liste non-exhaustive des résultats obtenus, on voit parfois la fréquence attendue de 50Hz 4 indiquant que le programme fonctionne correctement. Toutefois, À d'autres moments, il arrive d'avoir des fréquences telles que 35Hz 2 ou 90Hz 3 montrant que le stressé causé au CPU cause bien des délais dans l'envoi du signal.

4 Exercice 3 : Tâches périodiques à haute priorité sous Linux

Comparez cette nouvelle solution en termes de données statistiques avec la solution précédente.

Sans surcharge

```
evl-de1 ~ # ./pthread_timer 100
10033410
10026690
10024110
10024730
10024970
10022530
10022220
10022640
10021920
10021740
10022270
10021350
```

Avec surcharge

```
evl-de1 ~ # ./pthread_timer 100
10071800
10043410
10033180
10034380
10037770
10034630
10037480
10068880
10037750
10034960
10031780
10056520
10034290
```

Sans surcharge, la tâche périodique présente un comportement très stable, avec des périodes presque constantes autour de 10 ms. Sous surcharge, la variabilité augmente légèrement, mais l'effet reste limité. La politique d'ordonnancement `SCHED_FIFO` et la priorité élevée permettent donc d'améliorer significativement la régularité d'exécution par rapport à une tâche Linux classique.

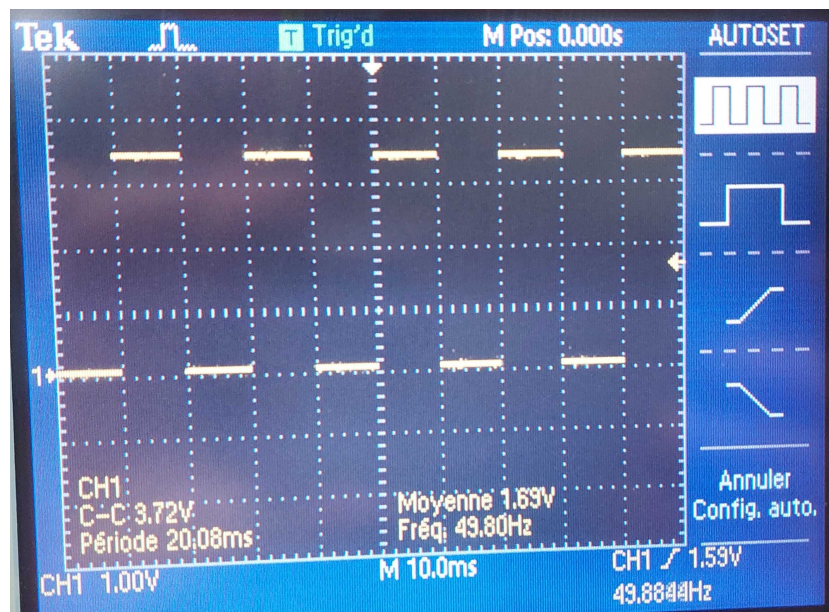


FIGURE 5 – Vue oscilloscope pthread_timer

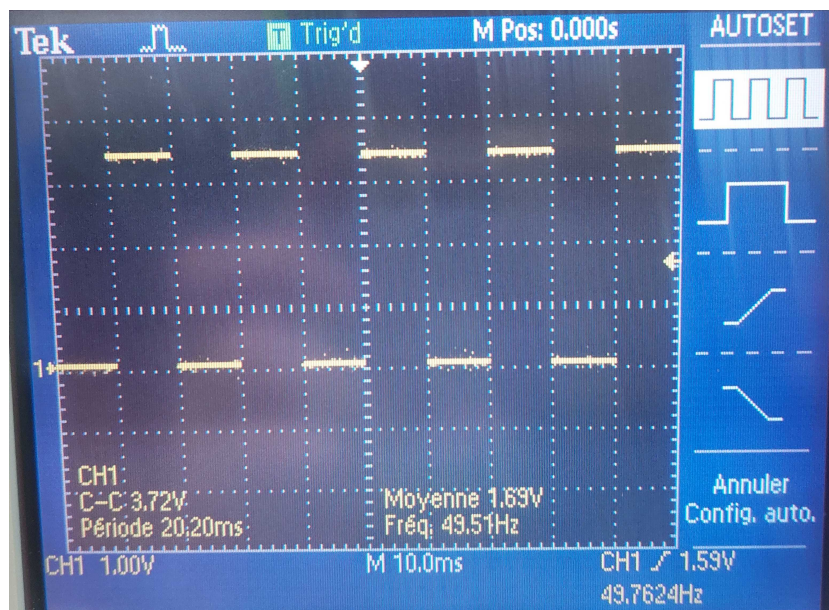


FIGURE 6 – Vue oscilloscope pthread_timer stressed

Nous pouvons également voir sur l'oscilloscope les mêmes comportements. Pour le cas sans surcharge 5, la tâche périodique reste stable. Sous surcharge 6, nous passons d'en moyenne 49.9Hz à un minimum de parfois 49.5Hz, ce qui est une nette amélioration comparé à l'exercice 2 3. On pourra remarquer que cela oscille entre 49.5Hz et 49.9Hz ce qui est un indéterminisme non négligeable.

5 Exercice 4 : Threads EVL

Ecrivez une nouvelle version de votre tâche périodique avec Xenomai EVL.

Sans surcharge

```
evl-de1 ~ # ./evl_timer 100
9993670
9999370
9999860
10000010
9999620
10000150
9999690
10000100
10000040
10000110
9999890
10000070
9999850
9999990
9999930
10000110
```

Avec surcharge

```
evl-de1 ~ # ./evl_timer 100
9985750
9997360
9999290
9999600
9999900
9999940
10000090
9999960
10000250
9999980
10004880
9998730
9997180
```

Le programme `evl_timer.c` reprend le principe de `pthread_timer.c`, mais en s'appuyant sur un timer EVL.

La fonction `mlockall()` est utilisée afin d'éviter les fautes de page durant l'exécution de la tâche temps réel.

La tâche est lancée dans un thread POSIX, puis attachée au planificateur EVL via `evl_attach_self()`.

Le thread est assigné au CPU 1, préalablement isolé, afin d'améliorer le déterminisme.

Un timer EVL est créé à l'aide de `evl_new_timer()` sur l'horloge monotone, puis configuré avec une période de 10 ms.

La fonction `oob_read()` permet d'attendre chaque occurrence du timer. À chaque occurrence, le temps est mesuré avec `evl_read_clock()` et une broche GPIO est basculée.

Les mesures obtenues avec EVL sont plus régulières que celles issues de `signal_timer2.c` ou `pthread_timer.c`, en particulier lorsque le système est fortement sollicité.

Le jitter est réduit et les écarts extrêmes sont moins marqués.

Cela s'explique par l'exécution de la tâche sur un CPU isolé, ainsi que par sa gestion via le planificateur temps réel EVL.

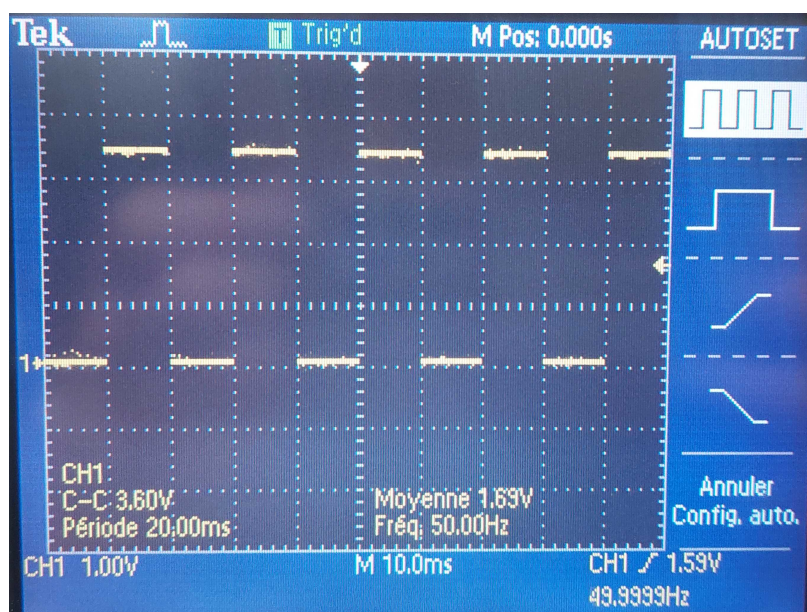


FIGURE 7 – Vue oscilloscope evl_timer

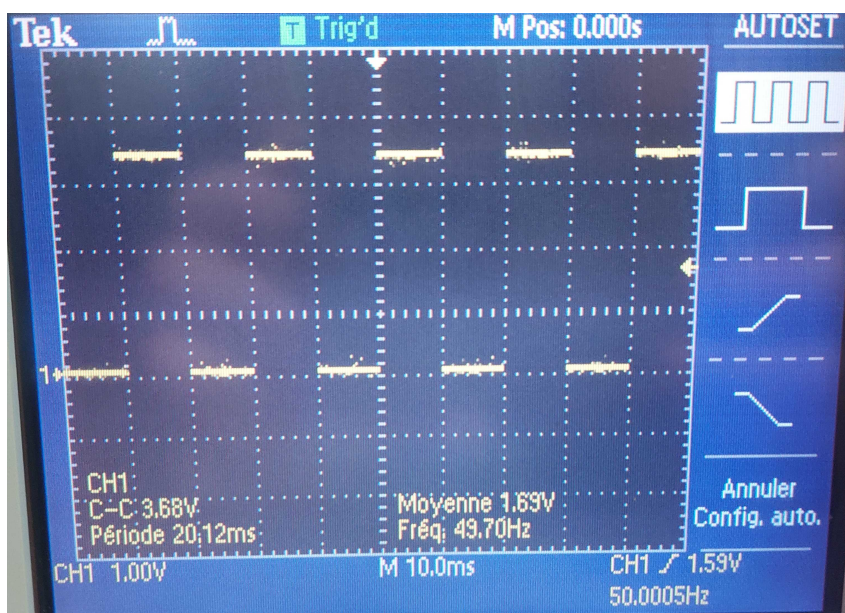


FIGURE 8 – Vue oscilloscope evl_timer stressed high

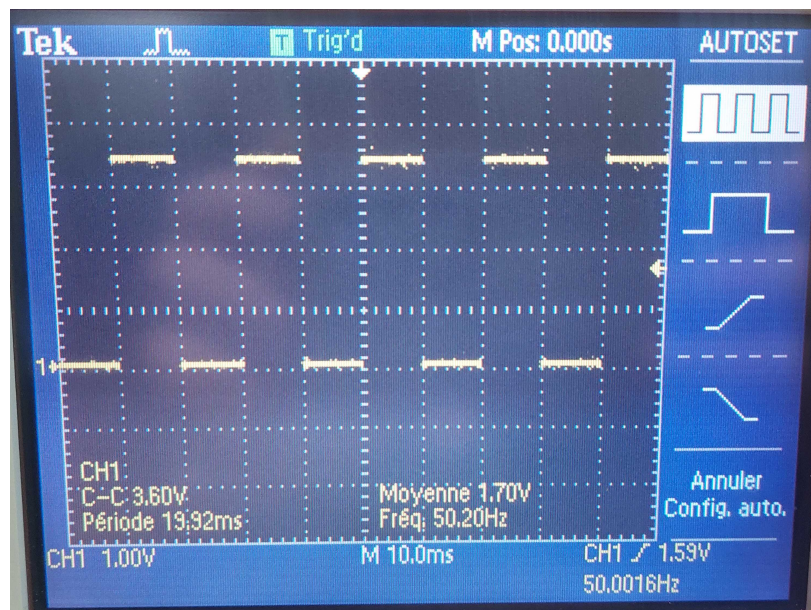


FIGURE 9 – Vue oscilloscope evl_timer stressed low

L'oscilloscope présente les mêmes résultats qu'en ligne de commandes. Lorsque le système n'est pas surchargé 7, la fréquence est stable à 50Hz, probablement dû à l'utilisation du coeur temps réel et de EVL. Lors d'une surcharge, le programme résiste bien en ayant un minimum autour de 49.7Hz 8 et un maximum de 50.2Hz 9. À la place d'être en dessous de notre valeur, nous oscillons à environ 50Hz. Cela ressemble au comportement de l'exercice 2 3 tout en ayant des performances meilleures que l'exercice 3 4.

6 Conclusion

Les résultats montrent que Linux classique présente des variations importantes sous charge. L'utilisation de SCHED_FIFO améliore la régularité, mais reste limitée. Le problème reste que la tâche "temps réel" est considéré comme les autres. Le CPU ne tient donc pas compte de l'échéance de celle-ci, car cette notion n'existe pas en temps normal.

Le thread EVL offre les meilleurs résultats avec un comportement plus stable et déterministe, notamment grâce à l'utilisation d'un CPU isolé et d'un planificateur temps réel. On remarque bien l'utilité d'EVL qui permet de donner une importance supérieure aux tâches temps réel. Il y a également un support pour celles-ci, avec une périodicité des tâches. On évite ainsi la surcharge due aux tâches Linux, car les tâches EVL sont exécutées en premier lieu. Puis, lorsqu'il y a le temps, EVL laisse les tâches Linux s'exécuter.

7 Signatures

Yverdon-les-Bains le 30 avril 2026

Fabien Leger & Arnaut Leyre